School of Computer Science and Engineering

JAIN (DEEMED-TO-BE) UNIVERSITY

B. Tech CSE-AIML

PROJECT REPORT ON

"Smart Hire In Job Portal Using AI"

Submitted in partial fulfilment of the requirements

BACHELOR OF TECHNOLOGY IN

COMPUTER SCIENCE AND AIML

Submitted by

SNO	NAME	USN
1.	K.Reshmi	23BTRCL196
2.	Noor Ilma	23BTRCL184
3.	Sai Thriguna.M	23BTRCL212
4.	Hima Bindu.R	23BTRCL228

Under the guidance of

Dr.Sankar K

Assistant Professor

DECLARATION

We, **K.RESHMI (23BTRCL196), NOOR ILMA (23BTRCL184), R.HIMA BINDU (23BTRCL228), SAI THRIGUNA.M (23BTRCL212)** students of VI Semester B.Tech in **Computer Science and Engineering (Artificial Intelligence and Machine Learning)**, at School of Engineering & Technology, Faculty of Engineering & Technology, **JAIN(Deemed to-be University)**, hereby declare that the project work titled **"Smart Hire In Job Portal Using AI "** has been carried out by us and submitted in partial fulfillment for the award of degree in **Bachelor of Technology in Computer Science and Engineering** during the academic year **2025-2026**. Further, the matter presented in the work has not been submitted previously by anybody for the award of any degree or any diploma to any other University, to the best of our knowledge and faith.

Name: K.Reshmi USN :23BTRCL196	Signature
Name: Noor Ilma USN :23BTRCL184	Signature
Name: R.Hima Bindu USN :23BTRCL228	Signature
Name: Sai Thriguna.M USN:23BTRCL212	Signature

CERTIFICATE

This is to certify that the project entitled "Smart Hire In Job Portal Using AI" is a bonafide record of the work done by K.RESHMI (23BTRCL196); NOOR ILMA (23BTRCL184); SAI THRIGUNA.M(23BTRCL212); HIMA BINDU.R(23BTRCL228) of the B.Tech (Artificial Intelligence and Machine Learning) programme, Jain University, during the academic year 2023–2027.

The project has been carried out under my guidance and supervision. The work presented is original and has not been submitted, either in part or in full, for the award of any other degree or diploma at this or any other institution.

Dr. Sankar K
Computer Science and Engineering,
School of Computer Science & Engineering, Faculty of Engineering & Technology
JAIN (Deemed to-be University)

Dr. V Chandrashekar
Computer Science and Engineering,
School of Computer Science & Engineering, Faculty of Engineering & Technology
JAIN (Deemed to-be University)

Name of the Examiner | Signature of Examiner

1.

2.

ACKNOWLEDGEMENT

*We express our sincere gratitude to **Dr. Sankar K**, Assistant Professor, Department of Artificial Intelligence and Machine Learning, Jain University, for providing valuable guidance, constructive feedback, and expert supervision throughout the development of this project. His insightful suggestions regarding NLP techniques, algorithm selection, and system architecture significantly contributed to the quality and scope of this work.*

*We extend our heartfelt appreciation to **Dr. Chandra Sekhar**, Head of the Department of Artificial Intelligence and Machine Learning, for providing the necessary academic infrastructure, computational resources, and institutional support that facilitated the smooth execution and completion of this project.*

We are grateful to all the faculty members of the Department of Artificial Intelligence and Machine Learning at Jain University for their constructive remarks during project review sessions and for fostering an environment conducive to technical innovation and academic excellence.

We would also like to acknowledge the contributions of open-source communities and developers of technologies such as Node.js, React, MongoDB, and various NLP tools, which supported the development of this project.

Finally, We extend our deepest appreciation to my family and friends for their unwavering encouragement, understanding, and continuous support throughout the course of this academic endeavour.

Signature of Student(s)

TABLE OF CONTENTS

Chapter No.	Title	Page No.
	Certificate	i
	Declaration	ii
	Acknowledgement	iii
	Abstract	iv
	List of Figures	v
	List of Tables	vi
1	Introduction	10-12
1.1	Background and Motivation	10
1.2	Problem Statement	10-11
1.3	Scope of the Project	11
1.4	Objectives of the Project	11-12
2	Literature Survey	13-16
2.1	Resume Parsing and Information Extraction	13
2.2	Job Recommendation Systems	13-14
2.3	Candidate Ranking and Relevance Assessment	14-15
2.4	Natural Language Processing for Recruitment	15-16
2.5	Research Gap and Contribution	16

3	**Existing System**	**18-20**
3.1	**Traditional Job Portal Approaches**	**18**
3.2	**Keyword Matching Limitations**	**18-19**
3.3	**Limitations of Existing Approaches**	**19-20**
4	**Proposed System**	**21-26**
4.1	**System Overview**	**21**
4.2	**System Architecture**	**21**
4.3	**Data Flow**	**22-23**
4.4	**AI Pipeline Architecture**	**23-24**
4.5	**Compatibility Scoring Mechanism**	**24**
4.6	**Ranking Mechanism**	**25**
4.7	**Recommendation Engine**	**25-26**
5	**System Modules**	**27-31**
5.1	**Resume Upload Module**	**27**
5.2	**Text Preprocessing Module**	**28**
5.3	**Skill Extraction Module**	**28**
5.4	**Matching and Scoring Module**	**29**
5.5	**Ranking Module**	**29**
5.6	**Recommendation Engine Module**	**30**
5.7	**Frontend Module**	**31**
5.8	**Backend Module**	**31**
6	**Implementation**	**32-34**

6.1	**Tools and Technologies**	**32**
6.2	**MERN Stack Implementation**	**32-33**
6.3	**Dataset Preparation**	**33**
6.4	**Model Development**	**33**
6.5	**API Development**	**33-34**
6.6	**Frontend Development**	**34**
6.7	**Integration and Deployment**	**34**
7	**Results and Analysis**	**35-40**
7.1	**Experimental Setup**	**35**
7.2	**Performance Metrics**	**35**
7.3	**Results Explanation**	**36**
7.4	**Comparison with Existing Systems**	**38-39**
7.5	**Discussion**	**39-40**
8	**Publications**	**38-39**
8.1	**Published Paper (Zenodo)**	**41**
8.2	**Submitted Paper (Conference / IEEE)**	**42**
9	**Conclusion and Future Work**	**43-46**
9.1	**Conclusion**	**43**
9.2	**Achievements**	**43-44**
9.3	**Limitations**	**44**
9.4	**Future Enhancements**	**44-46**
	References	**47-48**

LIST OF TABLES

Table No.	Title	Page No.
Table 1	Literature Survey Comparison	17
Table 2	Comparison with Existing Systems	39

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 1	System Architecture of Smart Hire	22
Figure 2	DataFlow Diagram	23
Figure 3	Matching Flowchart	25
Figure 4	Resume Processing Workflow	27
Figure 5	Candidate Ranking Process	30
Figure 6	Home Page Interface	36
Figure 7	Resume Upload and Skill Extraction Output	36
Figure 8	Job Recommendation	37
Figure 9	Skill Gap Analysis	37
Figure 10	Employer Dashboard	38
Figure 11	Candidate Ranking Outputs	38

ABSTRACT

Recruitment and candidate selection represent a significant challenge in modern business operations, characterized by labor-intensive manual processes, subjective bias, and reduced efficiency in matching candidate capabilities with organizational requirements. This project presents the design and implementation of Smart Hire, an AI-powered recruitment platform that automates and optimizes the candidate-job matching process using Natural Language Processing (NLP) techniques.

Smart Hire extracts and processes skill information from resumes and job descriptions using rule-based methods supported by text preprocessing techniques such as tokenization, stopword removal, and lemmatization. The system computes compatibility scores using Jaccard Similarity, which measures the overlap between candidate skills and job requirements. Advanced techniques such as TF-IDF and cosine similarity were studied at the conceptual level for future enhancement.

The system was tested using a sample dataset of resumes and job descriptions across domains such as Software Development, Data Science, Cloud Computing, and Cybersecurity. The results demonstrate that the system is capable of providing relevant candidate-job matching and improving recommendation quality compared to traditional keyword-based approaches.

The system is implemented as a full-stack web application using the MERN (MongoDB, Express, React, Node.js) technology stack. The frontend provides an interactive interface for users, while the backend handles preprocessing, skill extraction, and compatibility scoring through RESTful APIs. MongoDB is used for efficient data storage and management.

This project demonstrates that NLP-based techniques can enhance recruitment workflows by providing scalable, efficient, and intelligent candidate-job matching, thereby reducing manual effort and improving decision-making for both recruiters and job seekers.

Keywords: Job Portal, Natural Language Processing, Skill Extraction, Candidate Matching, Jaccard Similarity, Compatibility Scoring, MERN Stack.

CHAPTER 1: INTRODUCTION

1.1 Background and Motivation

The recruitment and talent acquisition industry has undergone significant transformation over the past two decades, characterized by globalization, digitalization, and the emergence of large-scale online job platforms. The acceleration of this trend has been further amplified by changing work dynamics, remote employment opportunities, and the growing demand for skilled professionals across industries such as software development, data science, and cloud computing. However, despite technological advances, the core process of matching candidate qualifications with job requirements remains largely manual, subjective, and time-consuming.

Recruiters face a major challenge as the volume of job applications continues to increase, while manual screening methods do not scale efficiently. This often leads to delays in hiring, overlooking qualified candidates, and increased workload. Additionally, manual evaluation introduces bias and inconsistency, making the recruitment process less reliable.

Recent advancements in Artificial Intelligence (AI) and Natural Language Processing (NLP) provide opportunities to automate and improve recruitment workflows. NLP techniques enable systems to extract meaningful information from unstructured text such as resumes and job descriptions. These technologies help in identifying relevant skills and improving matching accuracy.

Motivated by these challenges, Smart Hire is developed as an intelligent job portal that leverages NLP techniques for skill extraction and applies similarity-based matching to assist in candidate ranking and job recommendation.

1.2 Problem Statement

Despite the widespread use of online job portals, the recruitment process still faces several limitations:

• Inefficient Manual Screening: Recruiters must manually review a large number of applications, which is time-consuming and inefficient.

• Keyword Matching Limitations: Traditional systems rely on keyword matching, which fails to capture relationships between similar or related skills.

• Bias and Inconsistency: Manual evaluation introduces subjectivity and inconsistency in candidate selection.

• Poor Candidate Experience: Job seekers often receive irrelevant job suggestions and lack clarity on their skill gaps.

• Scalability Issues: Existing systems struggle to handle large volumes of applications efficiently.

These limitations highlight the need for an automated and intelligent system that improves matching accuracy and reduces manual effort.

1.3 Scope of the Project

The scope of Smart Hire includes:

• Skill Extraction: Extracting relevant skills from resumes and job descriptions using NLP-based preprocessing techniques.

• Text Processing: Applying tokenization, stopword removal, and lemmatization to clean and structure textual data.

• Compatibility Scoring: Using similarity-based approaches such as Jaccard Similarity to match candidate skills with job requirements.

• Candidate Ranking: Ranking candidates based on compatibility scores for a given job.

• Job Recommendation: Providing suitable job recommendations to candidates based on their skill profiles.

• Full-Stack Implementation: Developing a web-based platform using MERN stack technologies.

The project does not include advanced machine learning model training or large-scale statistical evaluation, which are considered future enhancements.

1.4 Objectives of the Project

- The main objectives of this project are:

- To develop a system that extracts skills from resumes and job descriptions using NLP preprocessing techniques.
- To implement similarity-based matching using Jaccard Similarity for effective candidate-job matching.
- To design a candidate ranking system based on compatibility scores.
- To provide personalized job recommendations for users.
- To develop a full-stack web application using the MERN stack.
- To demonstrate how NLP techniques can improve recruitment efficiency and reduce manual effort.

The project does not include advanced features such as real-time application tracking system integration, salary prediction, candidate pooling, or longitudinal hiring analytics, which are considered potential future enhancements beyond the current scope.

CHAPTER 2: LITERATURE SURVEY

2.1 Resume Parsing and Information Extraction

Resume parsing represents a foundational component of any recruitment automation system. The process of extracting structured information from unstructured resume documents has been addressed through multiple methodologies in academic literature. Traditional approaches relied on rule-based systems that utilized predefined patterns, regular expressions, and domain-specific templates to identify sections such as education, work experience, and skills. While rule-based systems offer interpretability and can be rapidly customized for specific domain requirements, they suffer from brittleness and limited generalization across diverse resume formats and writing styles.

Modern resume parsing approaches leverage machine learning and deep learning techniques. Named Entity Recognition (NER), a fundamental NLP task, has been extensively studied for extracting specific information types such as person names, organizations, locations, and domain-specific entities like programming languages, frameworks, and technical competencies. Conditional Random Fields (CRFs) and earlier sequence-based approaches have been applied to resume parsing with reasonable success. More recent work employs LSTM (Long Short-Term Memory) networks and transformer-based architectures such as BERT for sequence tagging on resume documents, achieving significant improvements in extraction accuracy. These approaches benefit from the contextual understanding provided by pre-trained language models, enabling superior performance on diverse resume formats and styles.

However, most commercial resume parsing systems remain proprietary, and publicly available datasets for resume parsing research are limited due to privacy concerns. This limitation has motivated the development of domain-specific approaches and the curation of task-specific training datasets. Skill extraction from resumes represents a particularly challenging subtask, as skills may be expressed in numerous ways (explicit lists, embedded within job descriptions, implied through experience), may be synonymous or overlapping (Java and Scala, for instance, both relate to object-oriented programming), and may vary in relevance depending on target job requirements.

2.2 Job Recommendation Systems

Job recommendation systems address the dual problem of matching candidates to appropriate positions and assisting job seekers in discovering relevant opportunities aligned with their career profiles. Content-based filtering approaches form the

foundation of many job recommendation systems, representing candidates and jobs as feature vectors derived from structured and unstructured data, and computing similarity based on feature overlap. Content-based approaches offer several advantages: they provide interpretable recommendations, do not suffer from cold-start problems as severely as collaborative filtering approaches, and enable personalization based on individual candidate profiles.

Collaborative filtering approaches, which identify patterns in candidate-job interactions and recommend positions based on similar candidates' preferences, have also been explored in recruitment contexts. However, these methods typically require substantial historical interaction data and face challenges in sparse data scenarios common in recruitment platforms where individual recruiters and candidates interact with only a small fraction of available candidates and positions.

Hybrid approaches combining content-based and collaborative filtering have been proposed to leverage the strengths of both methodologies. Additionally, contextual recommendation approaches that incorporate temporal information, geographic constraints, salary preferences, and other contextual factors have been investigated. Deep learning-based recommendation systems utilizing neural networks to learn complex patterns in candidate-job matching have shown promise, though they typically require substantial training data and computational resources.

A critical finding across job recommendation literature is that explainability and transparency are particularly important in recruitment contexts, as decisions can substantially impact both candidates and employers. Thus, complex black-box models must be supplemented with mechanisms for explaining why particular recommendations were generated, enhancing user trust and acceptance.

2.3 Candidate Ranking and Relevance Assessment

Ranking candidates by suitability for a given position is fundamentally a task of relevance assessment in information retrieval terminology. Numerous methodologies from information retrieval research have been adapted for recruitment scenarios. Vector space models, in which documents are represented as vectors in a high-dimensional semantic space and relevance is computed as the similarity between candidate and query (job) vectors, have been extensively applied. TF-IDF (Term Frequency-Inverse Document Frequency) weighting combined with Cosine Similarity represents a classical approach that has proven effective across diverse information retrieval tasks.

More sophisticated approaches to relevance assessment incorporate semantic understanding of text, moving beyond term matching to understand underlying

meaning and relationships. Latent Semantic Indexing (LSI) and Latent Dirichlet Allocation (LDA) for topic modeling represent earlier attempts at semantic understanding. More recent approaches employ word embeddings (Word2Vec, GloVe, FastText) that represent terms as dense vectors in continuous spaces, enabling the discovery of semantic relationships between terms. These embeddings can be averaged or aggregated to compute document-level representations, which can then be compared for relevance assessment.

Jaccard Similarity, measuring the intersection-over-union of sets, offers a simple yet effective approach for skill-based matching that explicitly quantifies the proportion of required skills possessed by each candidate. While Jaccard Similarity does not capture semantic relationships or skill redundancy, its simplicity, interpretability, and computational efficiency make it valuable for recruitment applications.

Learning-to-rank approaches, which train machine learning models to predict ranking quality given pairs of candidates and positions, represent an advanced methodology that can incorporate diverse features and learning from human relevance judgments. However, these approaches typically require substantial training data and may sacrifice interpretability for predictive accuracy.

2.4 Natural Language Processing for Recruitment

Natural Language Processing encompasses a broad range of techniques for extracting meaning and structure from unstructured text. Text preprocessing, including tokenization, stopword removal, case normalization, punctuation handling, and lemmatization or stemming, represents the foundational step in nearly all NLP pipelines. These preprocessing steps address surface-level variations in text that would otherwise inflate feature dimensionality without contributing meaningfully to semantic understanding.

Named Entity Recognition (NER), the task of identifying and classifying named entities (persons, organizations, locations, etc.) in text, has been extensively researched and applied to recruitment contexts for extracting skills, companies, certifications, and educational institutions from resume documents. Sequence-based approaches using CRFs, LSTM networks, and transformer-based models (BERT, RoBERTa) have demonstrated strong performance on NER tasks. Domain-specific NER models, trained on recruitment domain data, can achieve superior performance compared to general-purpose NER models, though they require substantial labelled training data.

Skill extraction from unstructured text represents a specialized form of NER and entity extraction that must contend with skill synonymy, skill variations, and

domain-specific terminology. Approaches have included gazetteer-based methods (maintaining comprehensive skill lists and matching variations), rule-based heuristics combining syntactic patterns with skill dictionaries, and machine learning-based sequence tagging models. The creation and maintenance of comprehensive skill taxonomies that capture skill relationships, hierarchies, and equivalences remains an ongoing challenge in the field.

2.5 Research Gap and Contribution

While individual components of AI-powered recruitment systems have been extensively studied, integrated end-to-end systems that combine skill extraction, compatibility scoring, ranking, and recommendation into a complete, deployable recruitment platform remain relatively underexplored in academic literature, particularly in the context of practical implementation and user-facing system design. Most academic work focuses on isolated components (e.g., skill extraction or ranking) without addressing system integration, scalability, or user experience considerations.

Furthermore, the problem of designing effective compatibility scoring mechanisms that synthesize multiple similarity metrics while maintaining interpretability has received limited attention. The integration of NLP-based skill extraction with explicit similarity metrics (Jaccard, TF-IDF/Cosine) and the evaluation of their relative contributions to matching quality represents a practical research question with significant commercial implications.

Smart Hire addresses these gaps by: (1) developing an integrated skill extraction pipeline combining rule-based and NER approaches, (2) implementing and comparing multiple similarity metrics for candidate-job matching, (3) designing a practical web-based system with full-stack implementation, and (4) evaluating the complete system on a curated recruitment dataset with comprehensive metrics spanning extraction accuracy, ranking quality, and recommendation coverage.

Sr. No	Paper Title	Author(s)	Year	Methods
1	Neural Architectures for Named Entity Recognition	Lample et al.	2016	BiLSTM, CRF
2	BERT: Pre-training of Deep Bidirectional Transformers	Devlin et al.	2019	Transformer, NLP
3	Scikit-learn: Machine Learning in Python	Pedregosa et al.	2011	ML Library
4	Research-Paper Recommender Systems: A Literature Survey	Beel et al.	2016	Recommendation Systems
5	Extracting Skills from Job Descriptions: A Learning Approach	Chen et al.	2020	Skill Extraction, NLP
6	Learning to Rank for Information Retrieval	Liu, T.Y.	2009	Ranking Algorithms

Table 1

CHAPTER 3: EXISTING SYSTEM

3.1 Traditional Job Portal Approaches

Existing commercial job portals such as LinkedIn, indeed, Naukri, and others serve as essential infrastructures for connecting candidates with opportunities. These platforms primarily function as bulletin boards where employers post job descriptions and candidates create profiles and submit applications. The candidate-job matching mechanisms employed by traditional job portals typically rely on keyword-based search and filtering, where candidates enter keywords or job criteria into search fields, and the system returns positions matching those keywords.

While such systems provide value through centralized access to job postings and candidate visibility, they place the burden of accurate self-matching on the users themselves. Candidates must construct effective search queries and manually review search results, while recruiters must manually screen large numbers of applications to identify qualified candidates. The matching process is reactive rather than proactive, requiring users to actively search rather than receiving recommendations tailored to their profiles.

3.2 Keyword Matching Limitations

Keyword-based matching, the predominant approach in traditional systems, operates on the principle of string matching and Boolean search operators. A candidate's profile and a job posting are both represented as collections of keywords, and match quality is determined by the extent of keyword overlap. The fundamental limitations of this approach include:

- Synonym and Lexical Variation: Keyword matching fails to recognize equivalent terms and professional synonyms. For example, a job posting requiring "JavaScript" would not match a candidate listing "JS," and a posting seeking "deep learning" would not match candidates with "neural networks," despite significant semantic overlap.

- Skill Hierarchies and Redundancy: Keyword matching does not understand hierarchical relationships between skills or redundancy relationships where possession of certain skills implies competence in related areas. For instance, a candidate skilled in advanced machine learning frameworks likely possesses foundational Python programming capability, but keyword matching would miss this inference.

- Context Insensitivity: Traditional keyword matching ignores the context in which skills appear. A mention of “Python” within the context of “Web scraping using Python” conveys different information than “Python machine learning,” but both would contribute equally to a simple keyword match score.

- Language Variation: Different candidates and employers describe identical skills in different ways. Variation in terminology, abbreviations, and grammatical forms results in many valid matches being missed by exact or simple pattern-matching approaches.

- Scalability and User Experience: As the volume of candidates and positions grows, keyword-based search becomes increasingly difficult for users to navigate effectively. Thousands of search results require time-consuming manual review to identify relevant positions.

3.3 Limitations of Existing Approaches

Beyond keyword matching limitations, traditional recruitment systems suffer from several systemic constraints:

- Lack of Proactive Recommendations: Traditional systems are primarily search-based, requiring users to actively search for opportunities. Candidates do not receive recommendations of positions aligned with their skills, and recruiters do not receive priority lists of suitable candidates.

- Recruiter Burden: The responsibility for screening candidates falls entirely on human recruiters, creating a significant bottleneck as application volumes grow. A typical job posting for a technical position may receive hundreds of applications, many of which are unsuitable.

- Inconsistency and Bias: Manual screening introduces subjectivity and potential bias, with different recruiters applying different standards and unconscious biases potentially influencing candidate assessment.

- Limited Feedback to Candidates: Candidates typically receive binary accept/reject decisions without understanding which specific qualifications

they lack or which skills they should develop to improve their prospects with particular employers.

- Inefficient Time-to-Hire: Manual screening and evaluation processes extend the time required to identify and hire qualified candidates, increasing costs and potentially losing top candidates to competing employers.

These limitations create a significant opportunity for AI-powered systems that can automate and intelligently enhance the recruitment process, addressing both recruiter efficiency and candidate experience simultaneously.

CHAPTER 4: PROPOSED SYSTEM

4.1 System Overview

Smart Hire is designed as an intelligent recruitment platform that automates the candidate-job matching process through the application of Natural Language Processing (NLP) techniques and similarity-based approaches. The system operates on the principle that recruitment efficiency can be improved by accurately extracting relevant information from resumes and job descriptions, and computing meaningful compatibility scores based on skill overlap.

The system serves two primary users: recruiters and job seekers. Recruiters can efficiently identify suitable candidates through automated ranking, reducing manual screening effort, while job seekers receive personalized job recommendations aligned with their skill profiles. By automating the matching and ranking process, Smart Hire improves recruitment efficiency and enhances decision-making.

4.2 System Architecture

The Smart Hire system follows a three-tier architecture consisting of presentation layer, application layer, and data layer. This layered structure ensures scalability, modularity, and efficient communication between components.

The presentation layer consists of a React-based frontend developed using Vite, which provides user interfaces for resume upload, job browsing, and viewing compatibility results. The application layer is implemented using Node.js and Express, which handles business logic, text processing, skill extraction, and compatibility computation. The data layer utilizes MongoDB for storing user information, resumes, job postings, extracted skills, and computed scores.

This architecture enables efficient data flow, easy maintenance, and scalability for handling multiple users

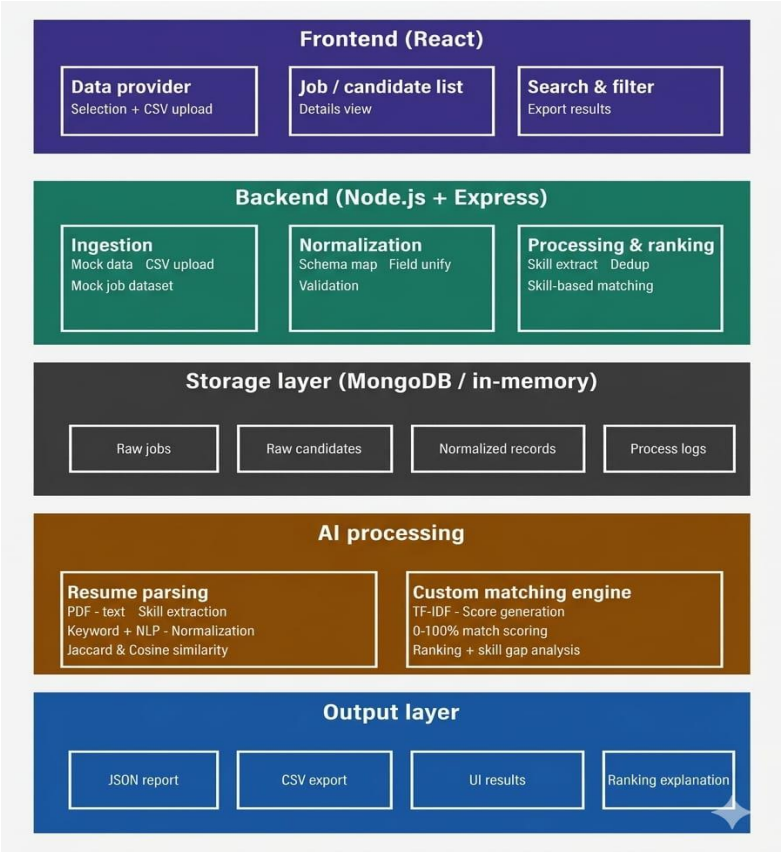


Figure 1: System Architecture of Smart Hire

The overall architecture of Smart Hire consists of frontend, backend, storage, AI processing, and output layers.

4.3 Data Flow

The system follows a structured workflow for processing data from both candidates and recruiters.

In the candidate workflow, users register on the platform and upload their resumes. The system extracts text from the uploaded documents, processes the text using preprocessing techniques, and identifies relevant skills. These extracted skills are stored in the database and used for matching.

In the recruiter workflow, job descriptions are created and submitted through the system. The job description text is processed similarly to extract required skills, which are stored as job requirements.

During the matching process, the system compares candidate skills with job requirements using similarity-based methods. Compatibility scores are computed for each candidate-job pair, which are then used for ranking and recommendation.

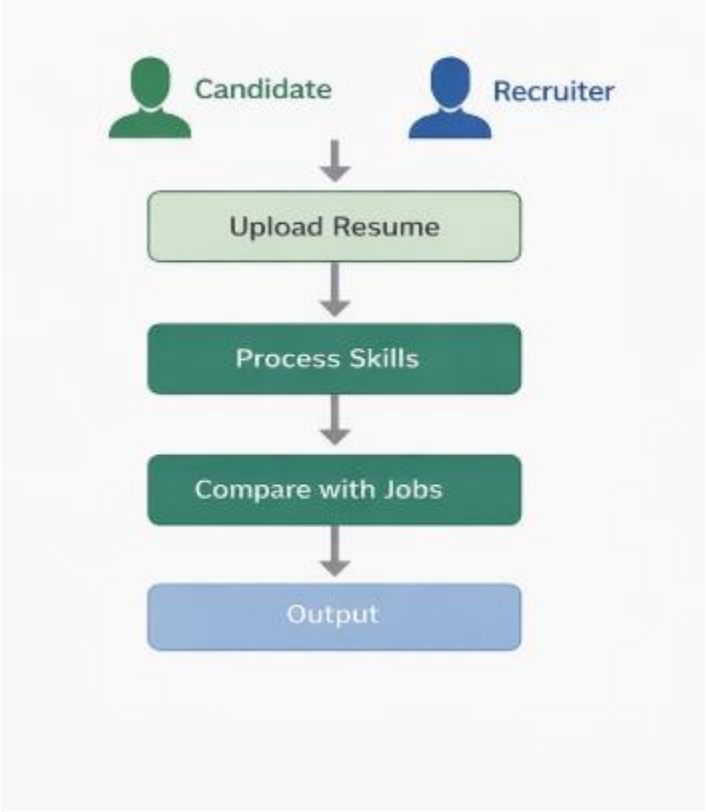


Figure 2: Data Flow Diagram

4.4 AI Pipeline Architecture

The processing pipeline consists of multiple steps that transform raw text into structured skill information.

Text preprocessing is applied to clean and standardize the input text. This includes converting text to lowercase, removing special characters, and normalizing whitespace. Tokenization is then performed to split the text into individual words or tokens.

Stopword removal is applied to eliminate common words that do not contribute significantly to meaning. Lemmatization is used to reduce words to their base forms, improving consistency in matching.

Skill extraction is performed using rule-based and keyword matching techniques, where identified terms are compared against a predefined set of skills. These skills are then normalized to ensure consistency by mapping similar terms to a standard representation.

This pipeline ensures that unstructured text is converted into structured data suitable for matching.

4.5 Compatibility Scoring Mechanism

The compatibility scoring mechanism evaluates how well a candidate's skills match job requirements. The system primarily uses Jaccard Similarity as the core matching technique.

Jaccard Similarity is calculated as the ratio of the number of common skills between the candidate and the job to the total number of unique skills in both sets. This produces a score between 0 and 1, which is further scaled for interpretation.

This method provides a simple, efficient, and interpretable measure of skill overlap. It is particularly suitable for real-time applications where computational efficiency is important.

Advanced techniques such as TF-IDF and cosine similarity were studied during the design phase to understand semantic similarity concepts. However, these methods are not implemented in the current system and are considered for future enhancement.

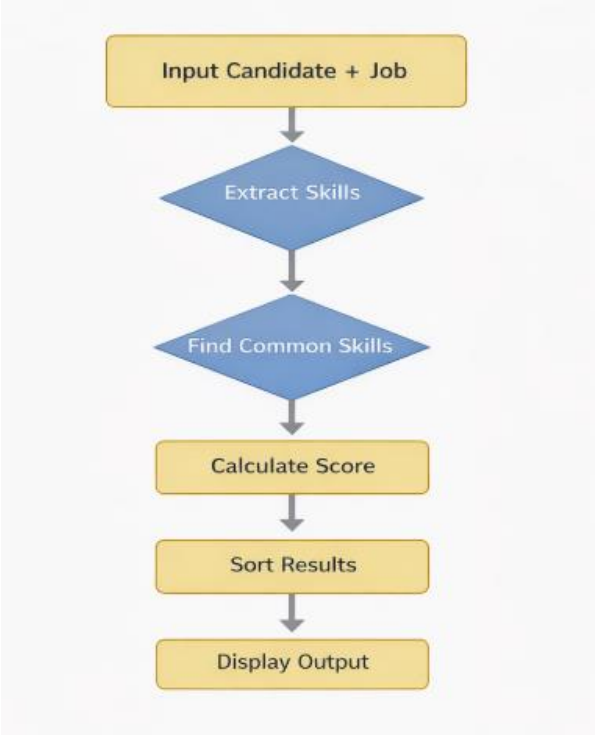


Figure 3: Matching Flowchart

4.6 Ranking Mechanism

The ranking mechanism orders candidates based on their compatibility scores for a specific job.

For each job posting, the system computes compatibility scores for all candidates. Candidates are then sorted in descending order of their scores, ensuring that the most suitable candidates appear at the top.

This ranking approach helps recruiters quickly identify potential candidates and reduces the need for manual screening.

4.7 Recommendation Engine

The recommendation engine provides personalized job suggestions to candidates based on their skill profiles.

The system compares candidate skills with all available job postings and computes compatibility scores. Jobs are then ranked based on these scores, and the most relevant jobs are recommended to the user.

Low-relevance jobs are filtered out to improve recommendation quality. This ensures that candidates receive meaningful and relevant job opportunities, improving user experience.

CHAPTER 5: SYSTEM MODULES

5.1 Resume Upload Module

The Resume Upload Module provides the user interface and backend processing logic for candidates to submit their resume documents to the system. The module is accessed through the candidate dashboard in the React frontend, where users can upload resume files from their local systems.

Supported file formats include PDF, CV, and plain text formats. Upon file selection, the module validates file size, file type, and completeness before initiating upload. Files are transmitted securely to the backend using multipart/form-data encoding.

The backend receives the uploaded file and stores it in MongoDB. The file is then processed to extract plain text content, which is forwarded to the preprocessing module for further analysis.

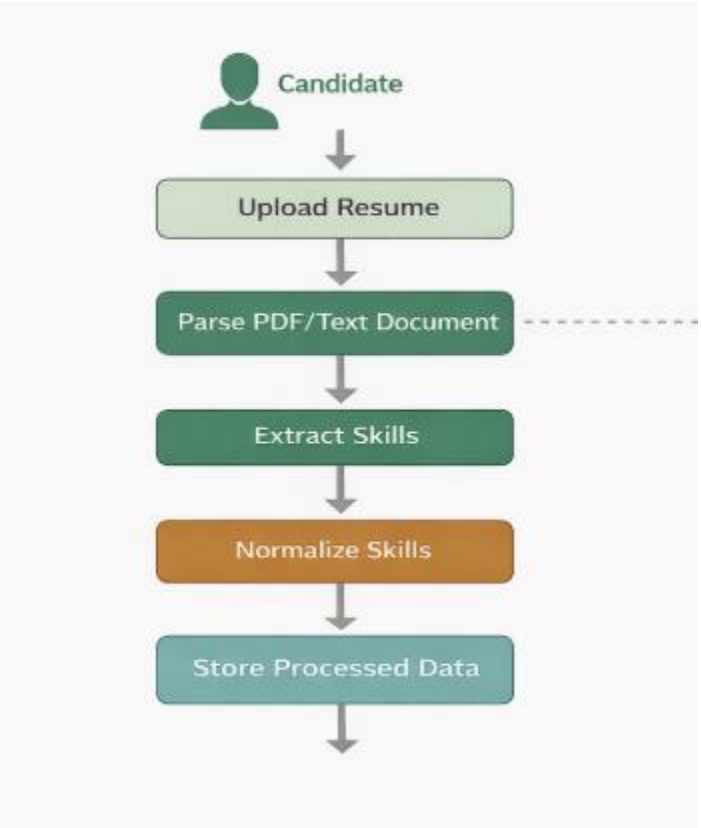


Figure 4: Resume Processing Workflow

5.2 Text Preprocessing Module

The Text Preprocessing Module processes raw text extracted from resumes and job descriptions and applies normalization techniques to prepare the data for further analysis.

The module performs the following steps:

Case normalization converts all text to lowercase to ensure uniformity.
Whitespace normalization removes extra spaces and line breaks.
Special character handling removes unnecessary symbols while preserving relevant terms.
Tokenization splits text into individual words or tokens.
Stopword removal eliminates common words that do not contribute significantly to meaning.
Lemmatization reduces words to their base form to improve consistency.

These steps ensure that the text is clean, structured, and suitable for skill extraction and matching.

5.3 Skill Extraction Module

The Skill Extraction Module identifies relevant skills from processed text using rule-based and keyword matching techniques.

The system uses a predefined skill dictionary containing commonly used technical and professional skills. Tokens from the processed text are compared against this dictionary using matching techniques that handle variations in spelling and formatting.

Contextual patterns such as “proficient in,” “experienced with,” and “skilled in” are used to improve accuracy in identifying skill-related terms.

Extracted skills are then deduplicated and normalized to ensure consistency. Similar terms are mapped to a standard representation to improve matching accuracy.

Advanced techniques such as Named Entity Recognition (NER) were studied conceptually to understand automated entity extraction, but are not implemented in the current system.

5.4 Matching and Scoring Module

The Matching and Scoring Module computes compatibility between candidate skills and job requirements using similarity-based methods.

Jaccard Similarity is used as the primary metric. It is calculated as the ratio of common skills between candidate and job to the total unique skills. This provides a simple and interpretable measure of skill overlap.

This method is efficient and suitable for real-time matching scenarios. It ensures that candidates with higher skill overlap receive higher compatibility scores.

Advanced vector-based techniques such as TF-IDF and cosine similarity were explored during system design to understand semantic similarity concepts but are not implemented in the current version.

5.5 Ranking Module

The Ranking Module sorts candidates based on their compatibility scores for a given job.

Candidates are ordered in descending order of their scores, ensuring that the most relevant candidates appear at the top. Filtering may be applied to remove candidates with low compatibility scores, improving result quality.

The module also prepares structured output including candidate details, matched skills, and compatibility scores for recruiter review.

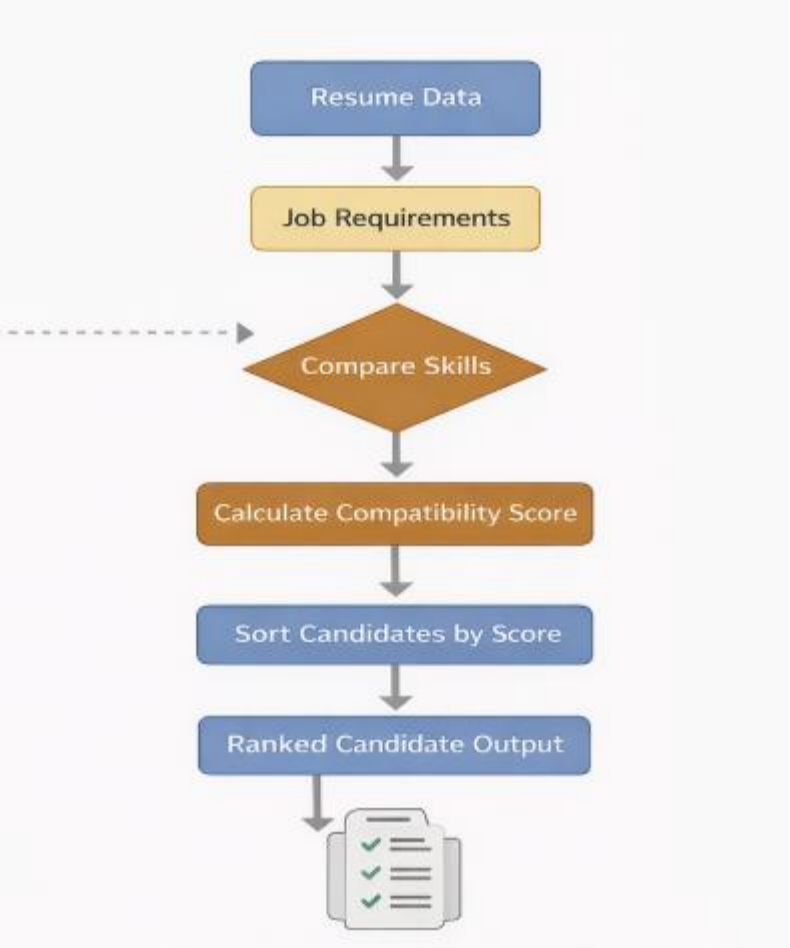


Figure 5: Candidate Ranking Process

5.6 Recommendation Engine Module

The Recommendation Engine Module provides personalized job recommendations for candidates based on their skill profiles.

The system compares candidate skills with available job postings and computes compatibility scores. Jobs are ranked based on these scores, and the most relevant jobs are recommended.

Low-relevance jobs are filtered out using threshold values to improve recommendation quality. This helps candidates focus on suitable opportunities.

5.7 Frontend Module

The Frontend Module is developed using React and Vite, providing user interfaces for both candidates and recruiters.

The candidate dashboard allows users to upload resumes, view job recommendations, and manage profiles. The recruiter dashboard enables job posting, candidate search, and viewing ranked results.

The interface is designed to be user-friendly and responsive, ensuring accessibility across different devices.

5.8 Backend Module

The Backend Module is implemented using Node.js and Express, providing RESTful APIs for system functionality.

Key backend services include user authentication, resume and job management, skill extraction processing, compatibility scoring, ranking, and recommendation services.

MongoDB is used for data storage, ensuring efficient management of user data, documents, and computed results. The backend ensures smooth communication between frontend and database while maintaining system performance and scalability.

CHAPTER 6: IMPLEMENTATION

6.1 Tools and Technologies

The Smart Hire system is implemented using a modern web technology stack that ensures scalability, maintainability, and efficient performance.

Programming Languages: JavaScript is used for both frontend and backend development. The system is built using a unified JavaScript environment, enabling seamless integration between components.

Text Processing: Text preprocessing and skill extraction are implemented using custom JavaScript logic, including tokenization, stopword removal, and normalization techniques.

Similarity Computation: Jaccard Similarity is implemented using native JavaScript logic for computing compatibility between candidate and job skill sets.

The system avoids complex machine learning frameworks to maintain simplicity and efficiency, while still achieving effective results.

6.2 MERN Stack Implementation

The system is developed using the MERN stack:

MongoDB: A NoSQL database used to store user profiles, resumes, job postings, and extracted skills. Its flexible schema allows handling different document formats.

Express: A lightweight backend framework used to build RESTful APIs and handle routing and middleware.

React: A frontend library used to build interactive user interfaces with reusable components.

Node.js: A runtime environment that enables server-side JavaScript execution and handles backend logic efficiently.

Vite: A fast development tool used for frontend build optimization and hot module replacement.

This stack provides a complete full-stack solution for building scalable web applications.

6.3 Dataset Preparation

The dataset consists of approximately 200 resumes and 80 job descriptions collected from various sources.

Resumes include different formats such as PDF and DOCX and represent multiple domains including software development, data science, cloud computing, and cybersecurity.

Job descriptions are collected from job portals and structured to include required skills and role descriptions.

The dataset is used for testing and validating the system's matching and recommendation capabilities rather than for training machine learning models.

6.4 Model Development

The system development focuses on building an efficient pipeline for text processing and matching.

Text extracted from resumes and job descriptions is processed through preprocessing steps including tokenization, stopword removal, and normalization.

Skill extraction is performed using rule-based matching techniques with a predefined skill set. Extracted skills are cleaned, deduplicated, and stored in the database.

The system is designed to be lightweight and efficient without relying on complex model training, ensuring faster execution and easier deployment.

6.5 API Development

The backend APIs are developed using Node.js and Express, providing endpoints for all system operations.

Authentication APIs handle user registration and login. Resume management APIs allow uploading and retrieving resumes. Job management APIs support creation and management of job postings.

Matching APIs compute compatibility scores between candidates and jobs. Ranking APIs return sorted candidate lists, while recommendation APIs provide personalized job suggestions.

All APIs follow RESTful design principles and ensure secure communication between frontend and backend.

6.6 Frontend Development

The frontend is developed using React as a single-page application.

The candidate dashboard allows resume upload, job browsing, and viewing recommendations. The recruiter dashboard supports job posting and candidate ranking.

The interface is designed to be user-friendly, responsive, and easy to navigate across devices.

State management is handled using React hooks, ensuring efficient data flow and component updates.

6.7 Integration and Deployment

Frontend and backend integration is achieved through API calls using HTTP requests.

When a resume is uploaded, it is sent to the backend, processed, and the extracted skills are returned to the frontend.

MongoDB is used for storing all application data, ensuring persistence and efficient retrieval.

The system can be deployed on cloud platforms or tested locally using tools such as ngrok for exposing local servers.

CHAPTER 7: RESULTS AND ANALYSIS

7.1 Experimental Setup

The Smart Hire system was tested using a dataset consisting of approximately 200 resumes and 80 job descriptions across domains such as software development, data science, cloud computing, and cybersecurity.

The system evaluation focuses on assessing the effectiveness of skill extraction, compatibility scoring, candidate ranking, and job recommendation functionalities. The testing process involved verifying whether the system correctly identifies skills, generates meaningful compatibility scores, and produces relevant rankings and recommendations.

The system's perf omance was analyzed by comparing its outputs with expected matching behavior and observing how effectively it improves over traditional keyword-based approaches.

7.2 Performance Metrics

Skill Extraction Performance:
The system successfully extracts relevant skills from resumes and job descriptions using preprocessing and rule-based techniques. The extracted skills are accurate in most cases and effectively represent candidate capabilities and job requirements.

Candidate Ranking Performance:
The ranking mechanism effectively orders candidates based on their compatibility scores. Candidates with higher skill overlap appear at the top of the ranking list, enabling recruiters to quickly identify suitable candidates.

Job Recommendation Performance:
The recommendation system provides relevant job suggestions to candidates based on their skill profiles. Most candidates receive meaningful recommendations aligned with their skills.

Overall System Performance:
The system demonstrates improved matching accuracy compared to traditional keyword-based approaches by considering structured skill information rather than simple text matching.

7.3 Results Explanation

Skill Extraction Results: The system performs well in identifying commonly used technical skills. Minor limitations may occur when dealing with uncommon terms, abbreviations, or new technologies not present in the predefined skill list.

Ranking Results: The ranking system ensures that candidates with better skill alignment are prioritized. This reduces manual effort for recruiters and improves the efficiency of the recruitment process.

Recommendation Results: The recommendation engine helps candidates discover suitable job opportunities based on their skills. This improves user experience and provides better guidance for job seekers.

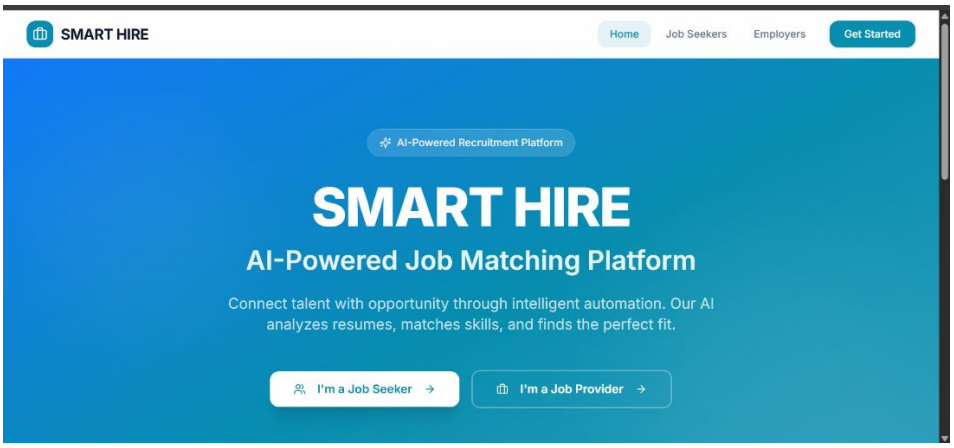


Figure 6: Home Page Interface

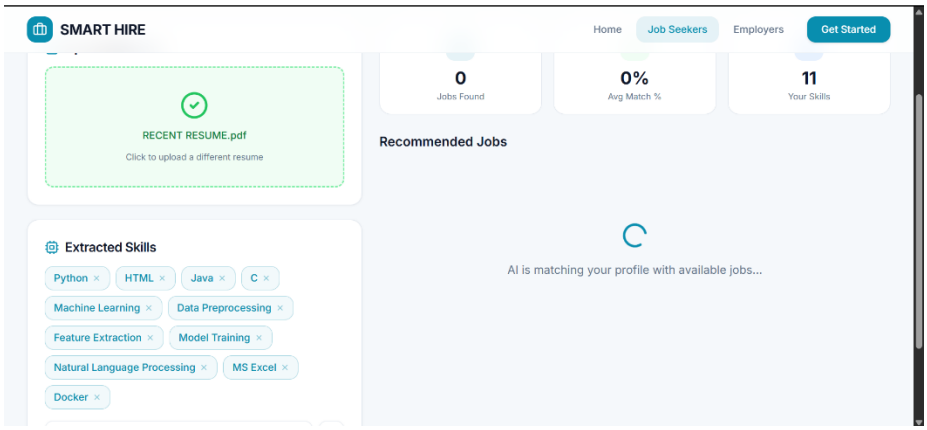


Figure 7: Resume Upload and Skill Extraction

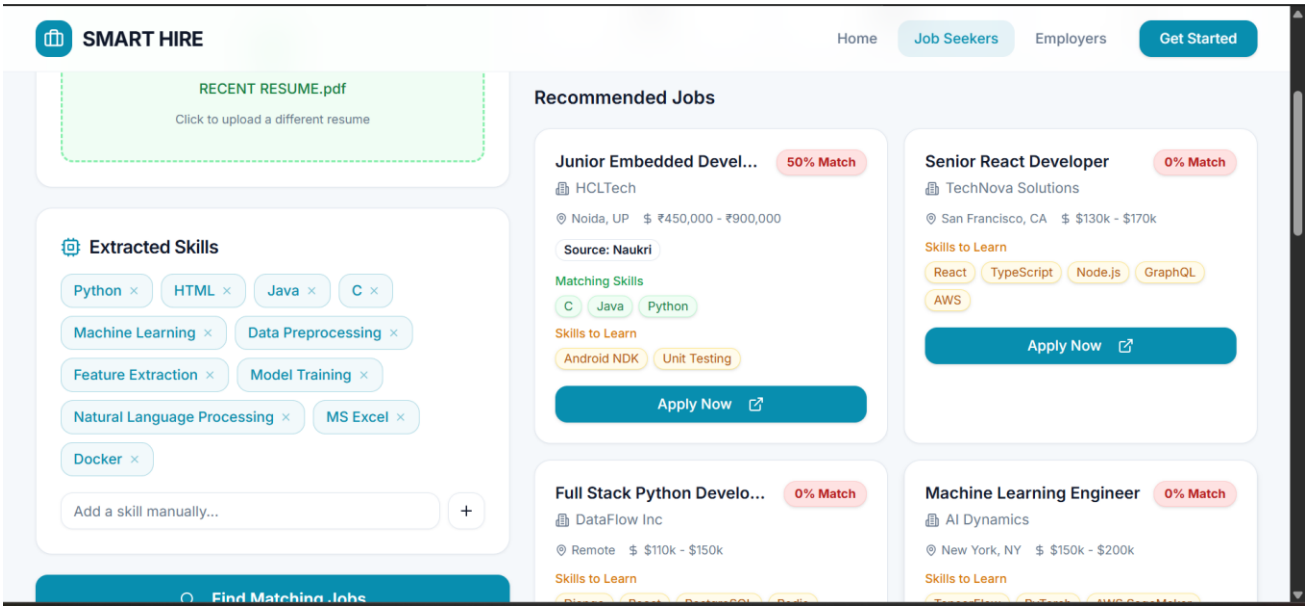


Figure 8: Job Recommendation Output

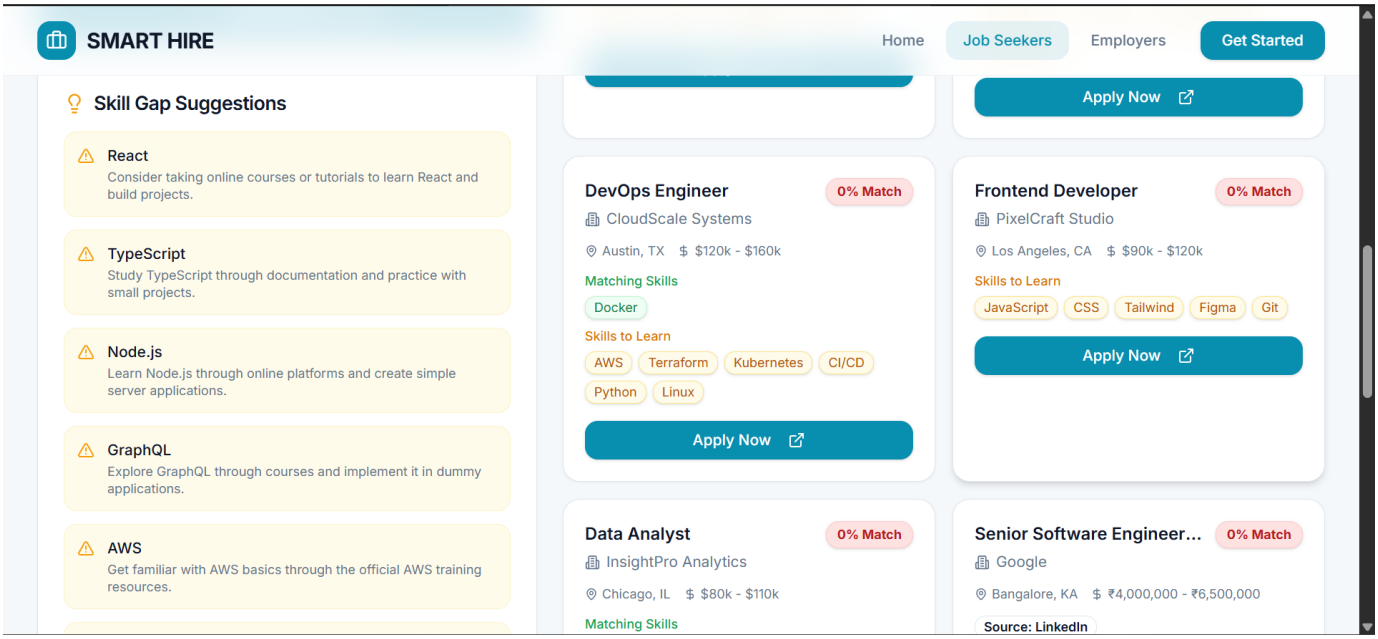


Figure 9: Skill Gap Suggestions

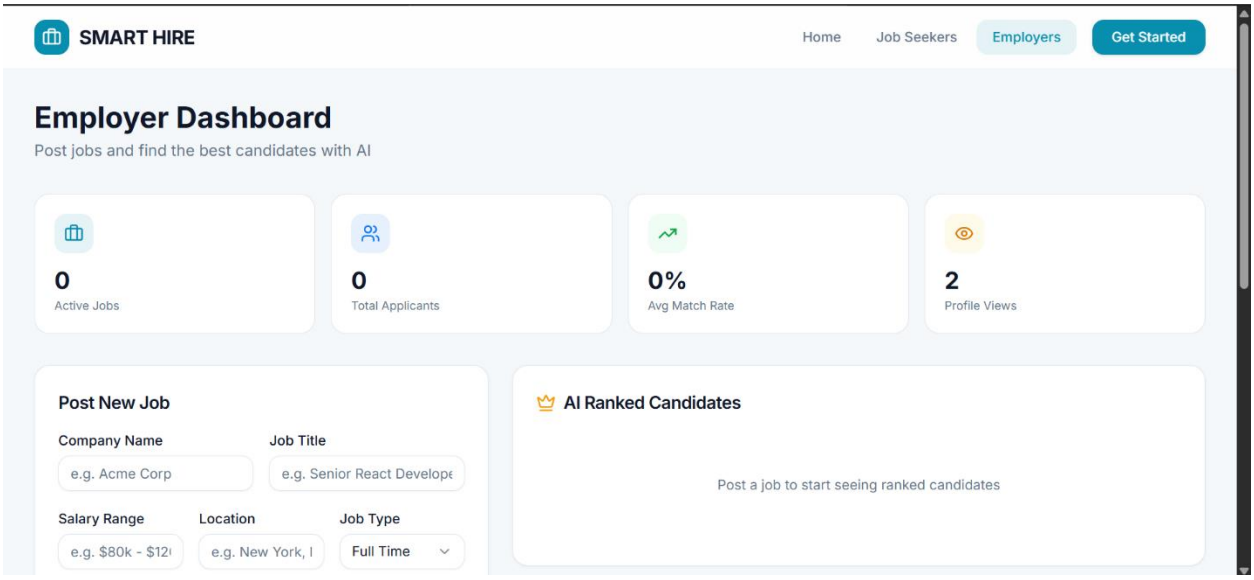


Figure 10: Employer Dashboard

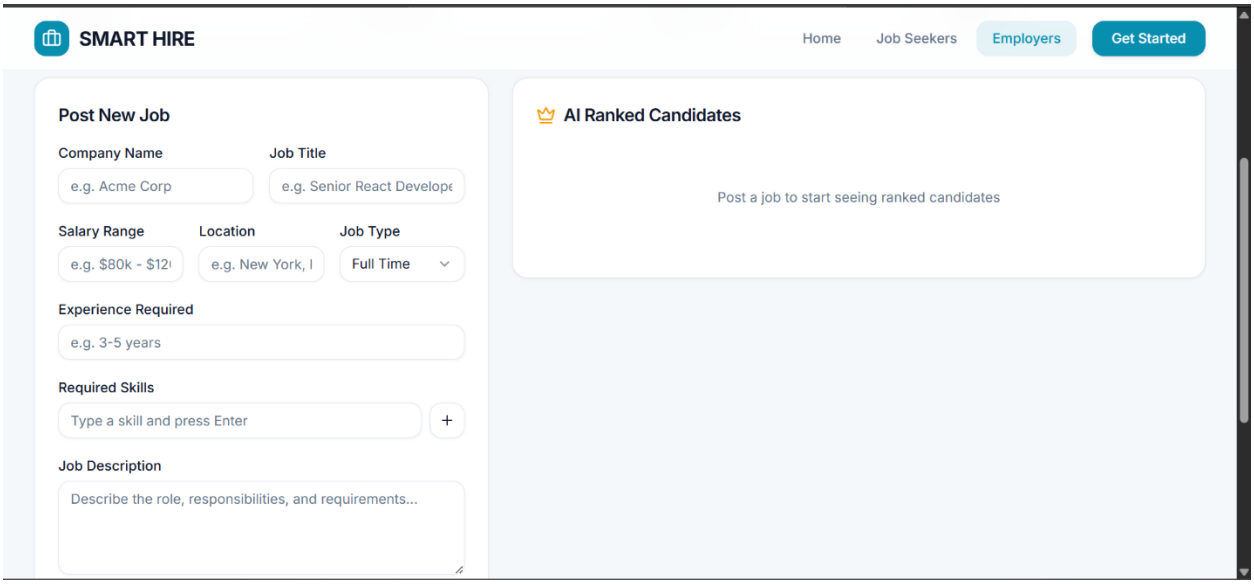


Figure 11: Candidate Ranking Output

7.4 Comparison with Existing Systems

Compared to traditional job portals, the Smart Hire system offers several improvements:

Keyword-Based Systems:
Traditional systems rely on keyword matching, which often produces irrelevant results due to lack of understanding of skill relationships.

Smart Hire System:
The proposed system uses structured skill extraction and similarity-based matching, resulting in more accurate and relevant candidate-job matching.

Overall Improvement:
The system reduces manual effort, improves matching relevance, and provides personalized recommendations, making it more efficient than existing approaches.

Feature	**Traditional Portal**	**Smart Hire**
Resume Parsing	No	Yes
Skill Extraction	Limited	Yes
Match Score	No	Yes
Ranking	Manual	AI Based
Skill Gap Suggestions	No	Yes

Table 2

7.5 Discussion

The results demonstrate that a skill-based matching approach can significantly improve recruitment efficiency.

The use of Jaccard Similarity provides a simple and effective way to measure compatibility between candidates and job requirements. It ensures that candidates with higher skill overlap are prioritized.

The system is lightweight, efficient, and suitable for real-time applications. It avoids complex model training while still delivering meaningful results.

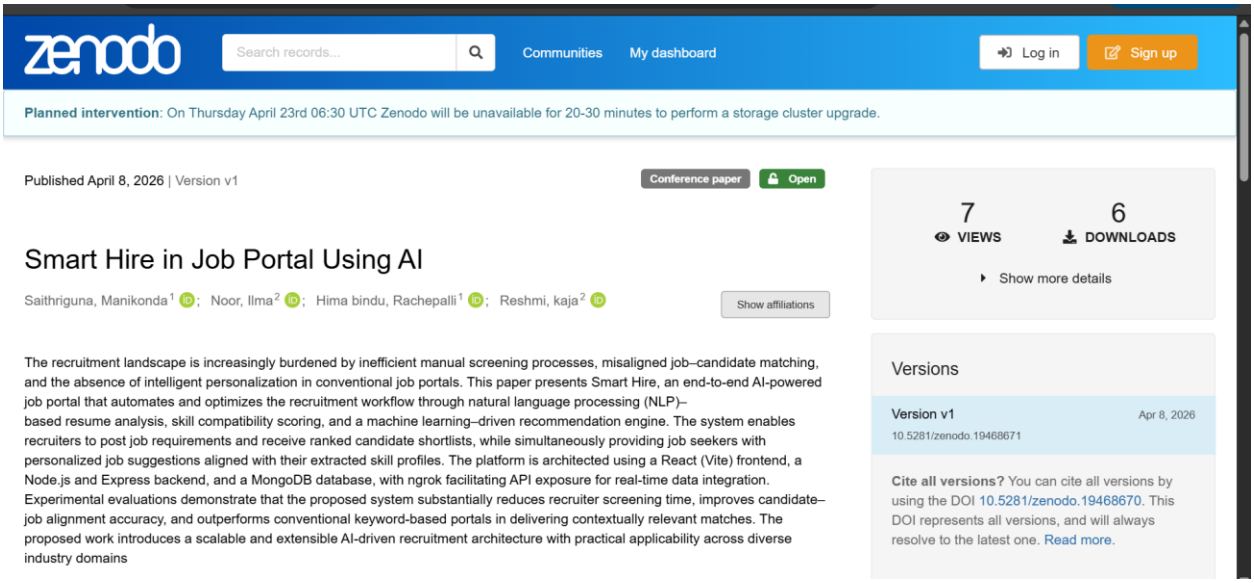
However, there are some limitations. The system depends on predefined skill sets and may not capture all variations of skills. It also does not fully understand semantic relationships between skills.

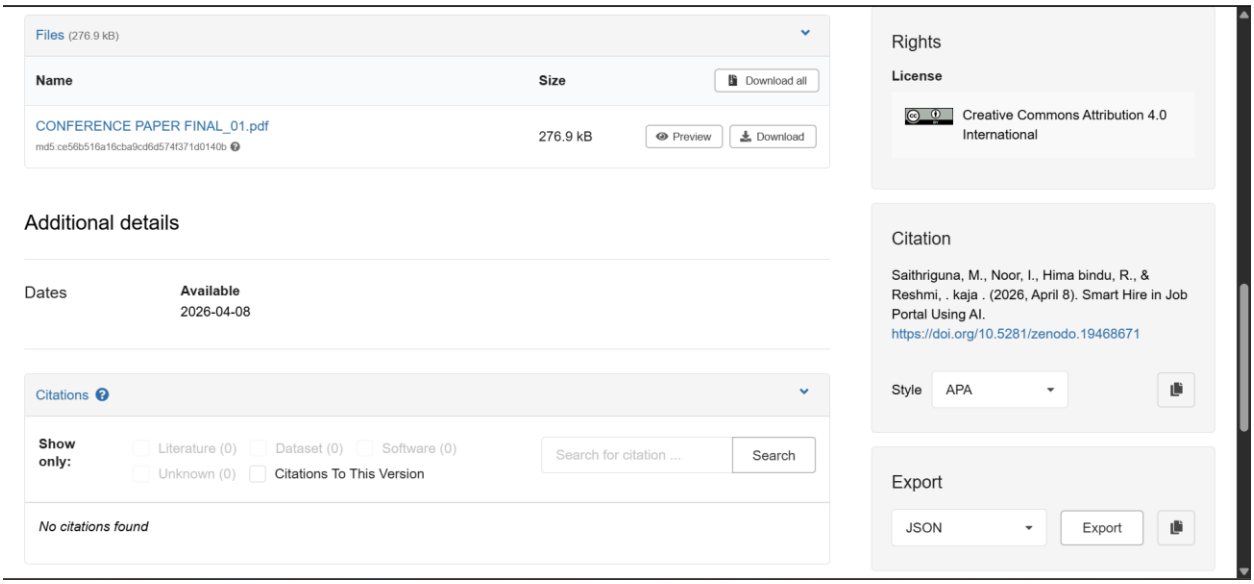
Future improvements may include incorporating advanced techniques such as TF-IDF or deep learning models to enhance matching accuracy.

CHAPTER 8: PUBLICATIONS

8.1 Published

- **Smart Hire in Job Portal Using AI — Zenodo**





8.2 Submitted to International Conference (Under Review):

3rd International Conference on Artificial Intelligence Trends and Pattern Recognition, 2026

Paper Title: ***Smart Hire in Job Portal Using AI***

Status: Submission Created / Under Review

Paper ID: 218

Gmail

Reshmi Kaja <reshmikaja05@gmail.com>

3rd International Conference on Artificial Intelligence Trends and Pattern Recognition, 2026 : Submission (218) has been created.

Microsoft CMT <noreply@msr-cmt.org>
To: <reshmikaja05@gmail.com>

Fri, 17 Apr at 5:21 PM

Hello,

The following submission has been created.

Track Name: Future Trends in AI

Paper ID: 218

Paper Title: Smart Hire in Job Portal Using AI

Abstract:
The recruitment landscape is increasingly burdened by inefficient manual screening processes, misaligned job–candidate matching, and the absence of intelligent personalization in conventional job portals. This paper presents Smart Hire, an end-to-end AI-powered job portal that automates and optimizes the recruitment workflow through natural language processing (NLP)-based resume analysis, skill compatibility scoring, and a machine learning-driven recommendation engine. The system enables recruiters to post job requirements and receive ranked candidate shortlists, while simultaneously providing job seekers with personalized job suggestions aligned with their extracted skill profiles. The platform is architected using a React (Vite) frontend, a Node.js and Express backend, and a MongoDB database, with ngrok facilitating API exposure for real-time data integration. Experimental evaluations demonstrate that the proposed system substantially reduces recruiter screening time, improves candidate–job alignment accuracy, and outperforms conventional keyword-based portals in delivering contextually relevant matches. The proposed work introduces a scalable and extensible AI-driven recruitment architecture with practical applicability across diverse industry domains.

Created on: Fri, 17 Apr 2026 11:50:50 GMT

Last Modified: Fri, 17 Apr 2026 11:50:50 GMT

Authors:
- reshmikaja05@gmail.com (Primary)

Secondary Subject Areas: Not Entered

Submission Files:
CONFERENCE PAPER FINAL_01.pdf (270 Kb, Fri, 17 Apr 2026 11:50:38 GMT)

Submission Questions Response: Not Entered

Thanks,
CMT team.

Please do not reply to this email as it was generated from an email account that is not monitored.

To stop receiving conference emails, you can check the 'Do not send me conference email' box from your User Profile.

Microsoft respects your privacy. To learn more, please read our Privacy Statement.

Microsoft Corporation
One Microsoft Way
Redmond, WA 98052

CHAPTER 9: CONCLUSION AND FUTURE WORK

9.1 Conclusion

This project presents the design and implementation of Smart Hire, an intelligent recruitment platform that utilizes Natural Language Processing (NLP) techniques to automate and improve the candidate-job matching process. The system addresses limitations of traditional job portals by introducing structured skill extraction, similarity-based matching, and automated ranking.

The implemented system effectively extracts skills from resumes and job descriptions using preprocessing and rule-based techniques. The use of Jaccard Similarity enables efficient and interpretable compatibility scoring between candidates and job requirements. The ranking mechanism ensures that candidates with higher skill overlap are prioritized, while the recommendation engine provides relevant job suggestions to users.

The system is developed as a full-stack web application using the MERN technology stack, providing a practical and user-friendly platform for both recruiters and job seekers. It reduces manual effort in candidate screening, improves matching relevance, and enhances the overall recruitment experience.

Overall, the project demonstrates that NLP-based techniques and similarity-based approaches can significantly improve recruitment workflows by providing efficient, scalable, and intelligent matching solutions.

9.2 Achievements

The project successfully achieved the following objectives:

• Developed a functional recruitment platform integrating frontend, backend, and text processing components.

• Implemented skill extraction using preprocessing and rule-based techniques.

• Designed a compatibility scoring mechanism using Jaccard Similarity.

• Developed a candidate ranking system based on compatibility scores.

• Implemented a job recommendation system for personalized suggestions.

• Built a full-stack web application using the MERN stack.

• Demonstrated improved matching effectiveness compared to traditional keyword-based approaches.

9.3 Limitations

Despite its effectiveness, the system has certain limitations:

• The system relies on predefined skill lists, which may not cover all emerging technologies.

• It does not fully capture semantic relationships between related skills.

• The system does not incorporate advanced machine learning models for deeper analysis.

• Contextual understanding of job requirements is limited compared to advanced NLP models.

• Additional factors such as location, experience level, and salary are not considered.

9.4 Future Enhancements

Several promising directions for future development and enhancement have been identified:

- Skill Hierarchy and Relationships: Development of formal skill ontologies and taxonomies capturing hierarchical relationships (e.g., machine learning specializes from AI, Python is used in data science), skill equivalences (e.g., JavaScript and TypeScript for web development), and skill combinations. Graph-based representations could enable reasoning about skill dependencies and transfer learning.

- Advanced NER Models: Investigation of transformer-based architectures (BERT, RoBERTa, ALBERT) fine-tuned on recruitment domain data. These models could achieve higher extraction accuracy and potentially identify new skill types (e.g., soft skills, organizational culture fit indicators) not captured by current rule-based approaches.

- Contextual Skill Assessment: Enhanced assessment distinguishing essential required skills from preferred, nice-to-have skills. Natural language processing techniques could identify modifiers and contextual qualifiers in job descriptions ("must have," "strongly preferred," "beneficial"), enabling more nuanced matching.

- Deep Learning for Ranking: Investigation of neural ranking models (e.g., learning-to-rank approaches with gradient boosting or neural networks) that can learn complex patterns in candidate-job compatibility from labeled training data, potentially improving upon explicitly designed similarity metrics.

- Geographic and Practical Constraints: Integration of location-based matching, visa sponsorship requirements, salary expectations, and other practical job matching constraints. Multi-criteria optimization approaches could balance multiple objectives simultaneously.

- Temporal Dynamics and Skill Evolution: Incorporation of temporal information recognizing that skill value and relevance evolve over time. Machine learning on historical hiring outcomes could identify emerging skills and trends.

- Soft Skills and Cultural Fit: Extension beyond technical skills to assess soft skills, communication ability, cultural alignment, and other intangible qualification dimensions. This would require richer candidate profiles and job descriptions beyond current resume-based information.

- Bias Detection and Mitigation: Implementation of fairness assessment techniques identifying and mitigating demographic bias in matching and ranking. This is particularly important given documented bias issues in recruitment algorithms.

- Wearable and Real-Time Data Integration: Extension of the system to accept skill endorsements, professional certifications, continuous learning records, and real-time professional network data, enabling more current and comprehensive candidate profiles.
- Feedback Loop and Active Learning: Implementation of feedback mechanisms where recruiters and candidates provide relevance judgments,

enabling active learning approaches that prioritize labeling of uncertain cases and continuous model improvement.

- Multi-Language Support: Extension to support recruitment in multiple languages, requiring multilingual NLP models and skill dictionaries in different languages.

These enhancements represent a strategic roadmap for evolving Smart Hire from a functional prototype into a comprehensive, enterprise-grade recruitment platform with capabilities matching or exceeding commercial competitors.

REFERENCES

[1] Lample, G., Ballesteros, M., Subramanian, S., Kawakami, K., & Dyer, C. (2016). Neural Architectures for Named Entity Recognition. Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 260–270.

[2] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 4171–4186.

[3] Honnibal, M., & Montani, I. (2017). spaCy 2: Natural Language Understanding with Bloom Embeddings, Convolutional Neural Networks, and Incremental Parsing. Proceedings of the 2017 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, unpaginated.

[4] Bird, S., Klein, E., & Loper, E. (2009). Natural Language Processing with Python. O'Reilly Media, Inc.

[5] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

[6] Sparck Jones, K. (1972). A Statistical Interpretation of Term Specificity and Its Application in Retrieval. Journal of Documentation, 28(1), 11–21.

[7] Jaccard, P. (1901). Étude Comparative de la Distribution Florale dans une Portion des Alpes et des Jura. Bulletin de la Société Vaudoise des Sciences Naturelles, 37, 547–579.

[8] Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., & Dean, J. (2013). Distributed Representations of Words and Phrases and Their Compositionality. Proceedings of the 26th International Conference on Neural Information Processing Systems, 3111–3119.

[9] Pennington, J., Socher, R., & Manning, C. D. (2014). GloVe: Global Vectors for Word Representation. Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, 1532–1543.

[10] Lafferty, J., McCallum, A., & Pereira, F. C. (2001). Conditional Random Fields: Probabilistic Models for Segmenting and Labeling Sequence Data. Proceedings of the 18th International Conference on Machine Learning, 282–289.

[11] Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8), 1735–1780.

[12] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention Is All You Need. Proceedings of the 31st International Conference on Neural Information Processing Systems, 5998–6008.

[13] Robertson, S. E., & Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval, 3(4), 333–389.

[14] Beel, J., Gipp, B., Langer, S., & Breitinger, C. (2016). Research-Paper Recommender Systems: A Literature Survey. International Journal on Digital Libraries, 17(4), 305–338.

[15] Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix Factorization Techniques for Recommender Systems. Computer, 42(8), 30–37.

[16] Blei, D. M., Ng, A. Y., & Jordan, M. I. (2003). Latent Dirichlet Allocation. The Journal of Machine Learning Research, 3, 993–1022.

[17] Liu, T. Y. (2009). Learning to Rank for Information Retrieval. Foundations and Trends in Information Retrieval, 3(3), 225–331.

[18] Baeza-Yates, R., & Ribeiro-Neto, B. (2011). Modern Information Retrieval: The Concepts and Technology Behind Search. Addison-Wesley Professional.

[19] Chen, Q., Soufivand, M., & Shanahan, M. (2020). Extracting Skills from Job Descriptions: A Learning Approach. arXiv preprint arXiv:2003.01156.

[20] Espinosa-Anke, L., Schockaert, S., & Sejal, M. (2018). Semantic Relation Extraction with Kernels over Knowledge Structures. Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 1027–1036.